

than $N \log N$; but since you have just done an N^3 procedure to get the eigenvalues, you can afford yourself this little indulgence.)

eigen_sym.h

```
void eigsort(VecDoub_IO &d, MatDoub_IO *v=NULL)
```

Given the eigenvalues `d[0..n-1]` and (optionally) the eigenvectors `v[0..n-1][0..n-1]` as determined by Jacobi (§11.1) or `tqli` (§11.4), this routine sorts the eigenvalues into descending order and rearranges the columns of `v` correspondingly. The method is straight insertion.

```
{
    Int k;
    Int n=d.size();
    for (Int i=0;i<n-1;i++) {
        Doub p=d[k=i];
        for (Int j=i;j<n;j++)
            if (d[j] >= p) p=d[k=j];
        if (k != i) {
            d[k]=d[i];
            d[i]=p;
            if (v != NULL)
                for (Int j=0;j<n;j++) {
                    p=(*v)[j][i];
                    (*v)[j][i]=(*v)[j][k];
                    (*v)[j][k]=p;
                }
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §8.4.
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[1]
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

11.2 Real Symmetric Matrices

As already mentioned, the optimum strategy in most cases for finding eigenvalues and eigenvectors is, first, to reduce the matrix to a simple form, only then beginning an iterative procedure. For symmetric matrices, the preferred simple form is tridiagonal.

Here is a routine based on this strategy that finds all eigenvalues and eigenvectors of a real symmetric matrix. It is typically a factor of about five faster than the Jacobi routine of the previous section. The implementations of the functions `tred2` and `tqli` that reduce the matrix to tridiagonal form and then find the eigensystem are discussed in the next two sections.

There are two user interfaces, implemented as two constructors. The first constructor is the usual one:

```
Symmeig s(a);
```

It returns the eigenvalues of a in descending order in $s.d[0..n-1]$. The normalized eigenvector corresponding to $d[k]$ is in the matrix column $s.z[0..n-1][k]$. Setting the default argument to `false` suppresses the computation of the eigenvectors:

```
Symmeig s(a,false);
```

If you already have a matrix in tridiagonal form, you use the other constructor, which accepts the diagonal and subdiagonal of the matrix as vectors:

```
Symmeig s(d,e);
```

Again, you can suppress the computation of eigenvectors by setting the default argument to `false`.

Here is the routine:

```
struct Symmeig {
```

Computes all eigenvalues and eigenvectors of a real symmetric matrix by reduction to tridiagonal form followed by QL iteration.

[eigen-sym.h](#)

```
    Int n;
    MatDoub z;
    VecDoub d,e;
    Bool yesvecs;
```

```
    Symmeig(MatDoub_I &a, Bool yesvec=true) : n(a.nrows()), z(a), d(n),
        e(n), yesvecs(yesvec)
```

Computes all eigenvalues and eigenvectors of a real symmetric matrix $a[0..n-1][0..n-1]$ by reduction to tridiagonal form followed by QL iteration. On output, $d[0..n-1]$ contains the eigenvalues of a sorted into descending order, while $z[0..n-1][0..n-1]$ is a matrix whose columns contain the corresponding normalized eigenvectors. If `yesvecs` is input as `true` (the default), then the eigenvectors are computed. If `yesvecs` is input as `false`, only the eigenvalues are computed.

```
    {
        tred2();                Reduction to tridiagonal form; see §11.3.
        tqli();                Eigensystem of tridiagonal matrix; see §11.4.
        sort();
    }
```

```
    Symmeig(VecDoub_I &dd, VecDoub_I &ee, Bool yesvec=true) :
        n(dd.size()), d(dd), e(ee), z(n,n,0.0), yesvecs(yesvec)
```

Computes all eigenvalues and (optionally) eigenvectors of a real, symmetric, tridiagonal matrix by QL iteration. On input, $dd[0..n-1]$ contains the diagonal elements of the tridiagonal matrix. The vector $ee[0..n-1]$ inputs the subdiagonal elements of the tridiagonal matrix, with $ee[0]$ arbitrary. Output is the same as the constructor above.

```
    {
        for (Int i=0;i<n;i++) z[i][i]=1.0;
        tqli();
        sort();
    }
    void sort() {
        if (yesvecs)
            eigsrt(d,&z);
        else
            eigsrt(d);
    }
    void tred2();
    void tqli();
    Doub pythag(const Doub a, const Doub b);
};
```